Project Structure Overview

The CampusCare project is organized as a monorepo containing two primary applications: a Node.js/Express Backend and an Expo/React Native Mobile Frontend.

1. Backend Structure (/backend)

The backend follows a modular, middleware-driven architecture.

* src/index.js: The server entry point where Express is initialized and middleware/routes are registered.

* **src/config/db.js:** Managed PostgreSQL connection pooling.

* **src/controllers/:** Contains the core business logic (e.g., issues.controller.js handles the ticket lifecycle).

* **src/routes/:** Maps API endpoints (e.g., /api/issues) to their respective controllers.

* **src/middleware/:** Handles Authentication (JWT) and Authorization (Role-Based Access Control).

* **src/migrations/:** Contains the SQL schema definitions for tables, ENUM types, and indices.

* **uploads/:** Local storage for images uploaded by users (tickets) and workers (completion proof).

2. Frontend Structure (/campusCare-mobile/campuscare-app)

The mobile app is built with Expo and React Native, using the

file-based Expo Router.

* **app/:** The routing hub.

    * **(auth)/:** Screens for login and registration.

    * **(tabs)/:** The main navigation tabs, including the reporting form (submit.tsx), specific dashboards for worker, admin, and manager (manage.tsx), and the user's personal dashboard.

    * **ticket/[id].tsx:** Dynamic route for viewing detailed ticket history and adding comments.

* **src/services/api.ts:** The bridge between the frontend and backend, managing all fetch requests.

* **src/context/AuthContext.tsx:** Manages global user state, tokens, and persistent login.
* **src/constants/Colors.ts:** Centralized theme configuration (GIU branding).
* **assets/:** Repository for images, icons, and branding materials.

3. Root Level

* **final SRS.pdf:** The master requirements document.
* **README.md:** Project setup and developer instructions.